從 ISA 契約到資料路徑與控制

Processor implementation 把 ISA 的抽象契約變成 bit-level state transition。ISA 規定指令看到什麼，microarchitecture（微架構）決定用多少 cycles、多少 functional units 與多少 power 達成它。

本章依序建立單週期 MIPS subset、五階 pipeline、hazard control、exception，最後擴到 multiple issue 與 speculative execution。每一步都要能說出資料由哪個 register 送到哪個 unit，以及錯誤的代價是什麼。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.1 設計問題與範圍

`datapath` 是搬運與運算資料的元件集合，`control` 依 instruction、ALU flags 與外部 request 產生 selector 和 enable signals。state 由 PC（program counter，目前指令位址）、register file 與 memory 等 clocked elements 保存。

本章教學 subset 包含：

1. memory reference：`lw`、`sw`。
2. arithmetic/logical：`add`、`sub`、`and`、`or`、`slt`。
3. control transfer：`beq`、`j`。

一般五階時序題採沒有 architectural delay slot 的簡化 subset，taken/misprediction 直接 squash 所有較年輕錯路徑指令；control hazard 小節另以 classic pre-Release-6 MIPS 的 one-delay-slot ISA 作對照。兩者的 flush 數、`$ra=PC+8` 與 wrong-path instruction count 不可互套。本章仍不是完整 MIPS；`addi`、byte load/store、multiply、floating point 與更完整 exception support 需要更多欄位、單元或狀態。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

從功能性到可實作契約

一個 instruction 的實作必須指定：

- source operands 與 destination architectural state。
- instruction 執行時 memory read/write 的數量。
- branch/jump target 的計算方式。
- exception 發生時哪些 register 或 memory write 可以保留。
- back-to-back instructions 對同一 register 的 read/write 關係。

例如 `lw rt, imm(rs)` 先以 `rs + sign_extend(imm)` 算 address，再於 MEM 取得 data，最後在 WB 寫 `rt`。若後一條指令立即讀 `rt`，硬體必須等 data 回來或使用正確的 forwarding path。

把 correctness 寫成 architectural state 的最終可見結果，而不是只看某張 waveform 是否看起來合理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2 Clocking 與 edge-triggered state

Combinational logic 沒有內部 clocked state；它的 output 會隨 input 傳播。Edge-triggered register 只在指定 clock edge 取樣，所以一個 cycle 的邏輯不得靠 feedback 穿過多個 state updates。

簡化週期約束可寫成

$$T_{\text{clk}} \geq T_{\text{cq}} + T_{\text{logic,max}} + T_{\text{setup}} + T_{\text{skew}}.$$

`T_cq` 是 clock edge 到 register output 有效的延遲，`T_logic,max` 是最長 combinational path，`T_setup` 是下一 register 的要求，`T_skew` 捕捉 clock 到達差異。hold constraint 由最快路徑限制，不能只用最慢路徑證明正確。

教材若寫「一個 gate 等於 1 cycle」是把時間抽象掉；實體電路驗證要看 process、voltage、temperature corner。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

4.3 Datapath 的積木

基本 datapath elements 包括 instruction memory、data memory、PC、adder、ALU、register file 與 multiplexor（多工選擇器）。

Register file 可同時讀兩個 registers、寫一個 register。Read 是 combinational；write 在 clock edge 發生。Separate instruction/data memories 讓 IF 與 MEM 在同一個 cycle 發生，但這是教學上的 Harvard-style model。真實 unified cache 可用更多 ports 或拆分 pipeline stages。

32-bit register file 有 32 個 architectural registers。`$zero` 的 read value 固定為 0，對它的 write 被忽略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ALU 與控制的分層

Main control 解讀 opcode，產生 `RegDst`、`ALUSrc`、`MemtoReg`、`RegWrite`、`MemRead`、`MemWrite`、`Branch`、`Jump` 等 signals。ALU control 再依 instruction class 與 R-type `funct` 欄位選 ALU operation。

以簡化 encoding 為例：

ALUOp	instruction class	ALU action
`00`	`lw/sw`	address addition
`01`	`beq`	subtraction
`10`	R-type	decode `funct`

控制輸出可由 truth table 或 ROM 實作。Decoder 的 propagation delay 也是 critical path 的一部分。

例如 R-type 使用 MemtoReg=0、RegDst=1、RegWrite=1，將 ALU 結果寫入 rd；sw 也用 ALU 算位址，但 RegWrite=0，結果送到資料記憶體位址端，由 MemWrite 啟用寫入。ALU operation 相同不代表目的地相同。

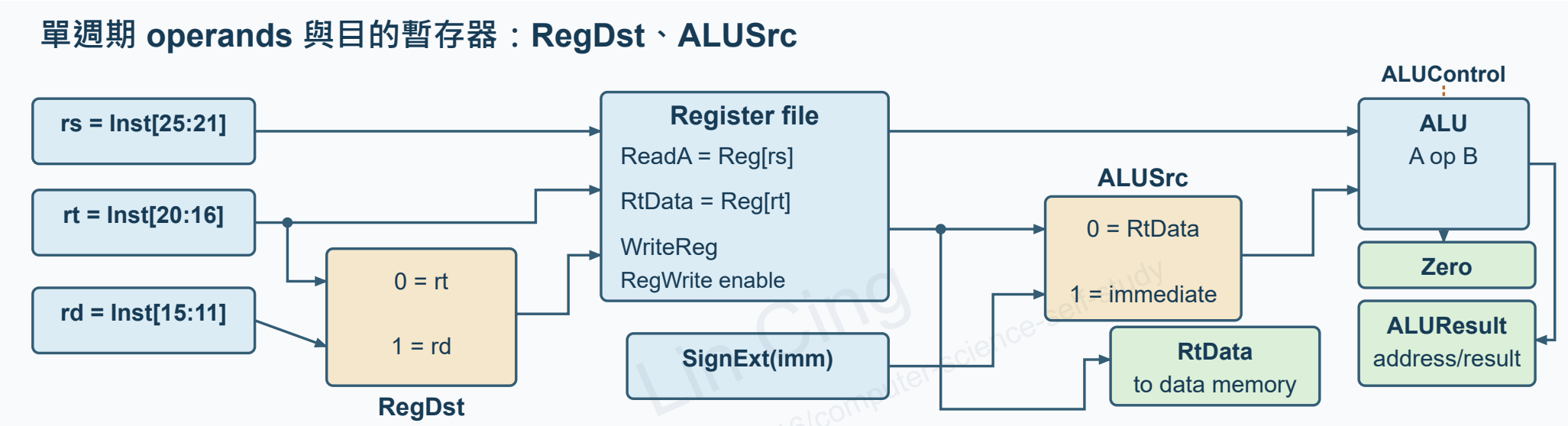
Single-cycle 的資料流

本節是無architectural delay slot的教學subset，支援R-type ALU、lw、sw、beq、j，獨立instruction memory與data memory；讀取為cycle內組合運算，PC與register writes、store在時脈邊緣提交。它沒有多週期的IR/A/B/MDR state，也沒有共用memory所需的IorD selector。

Cycle開始由PC.Q提供instruction address，instruction memory輸出opcode、rs、rt、rd、immediate與jump target fields。Control依opcode與funct產生select/enables；多條路徑同時運算，最後只有被enable的state更新，不是所有instruction都沿PC→IM→Reg→ALU→DM→WB一條直鏈走到底。

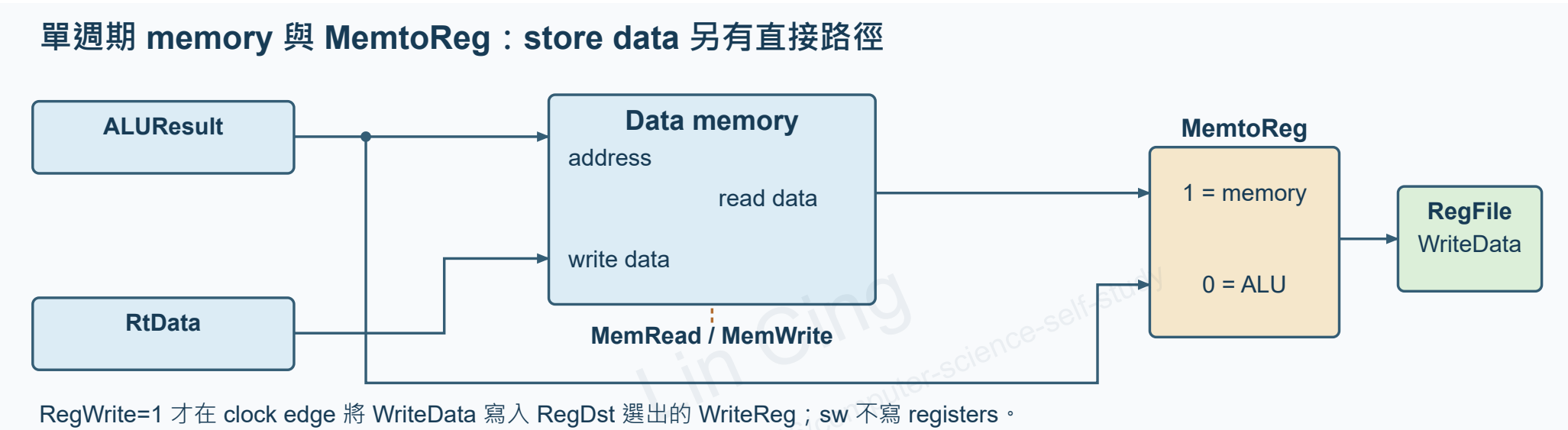
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Single-cycle 的資料流 （續）



RegDst=0選Instruction[20:16]的rt作WriteReg，RegDst=1選Instruction[15:11]的rd。ReadA=Reg[rs]、RtData=Reg[rt]。ALUSrc=0把RtData送進ALU第二輸入，=1改送sign-extended immediate；第一輸入是ReadA。RtData另外直接供sw的memory write data，不能因ALUSrc=1就把immediate當作要儲存的資料。

單週期 memory 與 write-back 的分支



ALUResult對lw/sw是有效位址，對R-type是運算結果。Data memory的write data來自RtData，只有MemWrite=1才在本cycle的提交邊緣寫入。MemRead=1時提供該位址資料給MemtoReg的memory輸入。

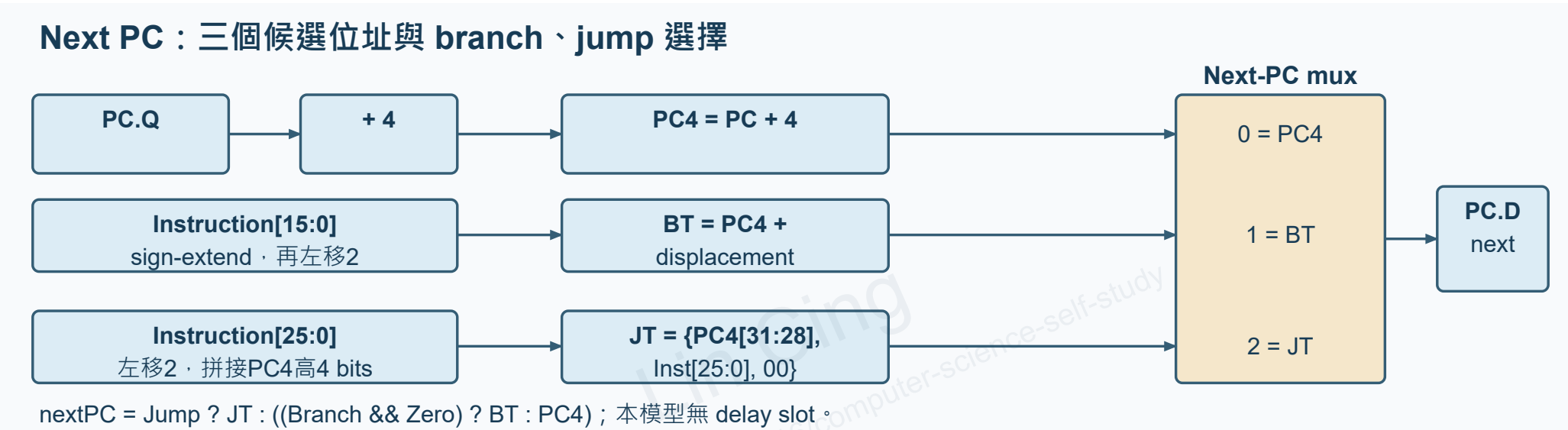
MemtoReg=0選ALUResult，=1選memory read data，形成WriteData。只有RegWrite=1，才把WriteData寫入上一圖RegDst選出的WriteReg；目的為$zero時依ISA不改其固定值。圖中相同名稱是同一組資料wire的跨圖端點，不代表額外pipeline registers。

單週期 memory 與 write-back 的分支 (續)

例如 `sw $t2,-4($s0)`，Reg[s0]=0x1000、Reg[t2]=0x12345678，immediate=-4。ALUSrc=1得到address0x0ffc，但store data仍為0x12345678，MemWrite=1、RegWrite=0。若誤把ALU的第二輸入當store data，就會把−4寫入memory。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單週期 next-PC 的三個候選



PC4=(PC+4) mod2^32。Branch displacement=(signext(imm)<<2) mod2^32，BT=PC4+displacement；beq以ALU做ReadA−RtData，Zero表示兩個32-bit值相等。Jump的JT拼接PC4[31:28]、Instruction[25:0]、00，不使用PC原值的高4 bits。

單週期 next-PC 的三個候選 （續）

選擇函數為 `nextPC = Jump ? JT : ((Branch && Zero) ? BT : PC4)`，可實作成先branch/seq、再jump的兩個2-input mux。下個edge把nextPC提交到PC；本模型不先執行delay slot。圖把相同PC4用名稱接到各計算式，所有計算都屬同一cycle的組合邏輯。

PC=0x00400020、beq immediate=-2時，PC4=0x00400024、BT=0x0040001c；相等則選BT，否則PC4。PC=0x0ffffffc且jump field=0時，PC4=0x10000000，所以JT=0x10000000；若錯用PC高bits會跳到0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Control table 與每個 mux 的選擇

Signal	lw	sw	R-type	beq	j
RegDst	0	X	1	X	X
ALUSrc	1	1	0	0	X
MemtoReg	1	X	0	X	X
RegWrite	1	0	1	0	0
MemRead	1	0	0	0	0
MemWrite	0	1	0	0	0
Branch	0	0	0	1	0
Jump	0	0	0	0	1
ALUOp	00	00	10	01	X

Control table 與每個 mux 的選擇 (續)

ALUOp=00選add用於address，01選subtract以產生Zero，10再由R-type funct選add/sub/and/or/slt等。X是don't-care，只在結果不會被提交的欄位使用；RegWrite/MemWrite不能因其他欄位不用而隨意assert。非法opcode與例外還需額外控制，本subset不假裝已涵蓋完整MIPS。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五類 instruction 的完整 state effects

取每一列獨立的cycle，PC=0x00400020、Reg[s0]=0x1000、Reg[s1]=7、Reg[s2]=5、Reg[t2]=0x12345678，Mem[0x100c]=0xabcdef01。Word accesses對齊，運算不產生本subset未處理的overflow。

Instruction	關鍵資料路徑	本cycle提交
addu t0,s1,s2	ReadA7、RtData5；ALUSrc0；ALU12；MemtoReg0；RegDst1	t0=12，PC=0x00400024
lw t1,12(s0)	ReadA0x1000、imm12；ALU0x100c；MemtoReg1；RegDst0	t1=0xabcdef01，PC=0x00400024
sw t2,-4(s0)	ALU0x0ffc；RtData0x12345678直入memory	Mem[0x0ffc]=0x12345678，PC=0x00400024
beq s1,s2,-2	ALU7−5=2，Zero0；Branch1，Jump0	無register/memory writes；PC=0x00400024
j 0x00401000	field0x00100400；Jump1	無register/memory writes；PC=0x00401000

五類 instruction 的完整 state effects （續）

若只把beq輸入改成兩者皆5，Zero=1，PC改為0x0040001c。Lw的rt欄位是目的暫存器，sw/beq的rt則是來源；不能把相同欄位名稱當作永遠同一讀寫用途。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單週期最慢路徑決定共同時脈

lw通常走PC clock-to-Q、instruction read、register read、ALU address、data-memory read、MemtoReg及register setup，因此

$$T_{clk} \geq T_{PCQ} + T_{IM} + T_{regread} + T_{ALU} + T_{DM} + T_{mux} + T_{setup}.$$

明定這些已合併的delay項為30+200+120+180+250+20+40 ps，得到840 ps。R-type的此模型路徑30+200+120+180+40=570 ps，仍需等待同一840 ps clock。這個簡化delay預算忽略的wire/control/mux成本必須已可被所用項目涵蓋，不能再聲稱是某個實際晶片的完整timing closure。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單週期效能與限制

以下是與前頁 840 ps 延遲預算不同的獨立模型，假設 clock period 為 800 ps，程式有 60% R-type、25% load/store、15% branch/jump，且 single-cycle CPI 對每條 instruction 都是 1。CPU time 為

$$T = IC \times CPI \times T_{\text{clk}} = IC \times 1 \times 800\,\text{ps}.$$

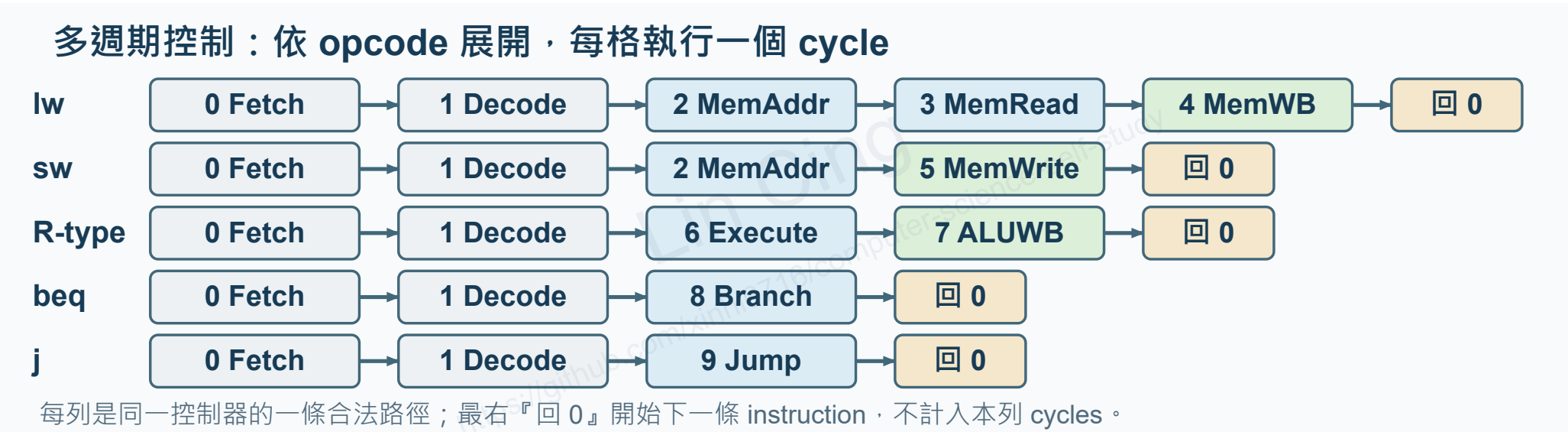
最佳化只縮短 R-type path 不會降低 clock period，因為 `lw` 仍決定週期。反之，任何只增加 `lw` path 的 feature 會拖慢所有 instructions。

Single-cycle 適合小教學 ISA，實際處理器常用 pipelining、multi-cycle control 或 asynchronous/clock-gated 方法控制資源與時序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多週期資料路徑與控制

多週期設計把 instruction 拆成若干 state，每個 clock edge 只讓一組資料動作的結果進入暫存 state。暫存器 `IR`、`A`、`B`、`ALUOut` 與 `MDR` 分別保存 instruction、兩個 register operands、ALU result 與 memory data；`PC` 仍然保存 architectural instruction address。



多週期資料路徑與控制 （續）

Instruction	State path	Cycles
R-type	Fetch → Decode → Execute → ALUWB → Fetch	4
`lw`	Fetch → Decode → MemAddr → MemRead → MemWB → Fetch	5
`sw`	Fetch → Decode → MemAddr → MemWrite → Fetch	4
`beq`	Fetch → Decode → Branch → Fetch	3
`j`	Fetch → Decode → Jump → Fetch	3

表中每一個 state 各占一個 cycle；最後的 Fetch 是下一條 instruction 的第一個 cycle。`sw` 從 MemAddr 直接到 MemWrite，不經 MemRead。

State	本次 clock edge 保存的資料或提交的效果	關鍵 write enable／選擇
0 Fetch	IR←Memory[oldPC]；PC←oldPC+4	IorD=0；MemRead=1；IRWrite=1；PCWrite=1

多週期資料路徑與控制 （續）

State	本次 clock edge 保存的資料或提交的效果	關鍵 write enable／選擇
1 Decode	A←Reg[rs]；B←Reg[rt]；ALUOut←PC+branch displacement	不寫 architectural registers／memory

State	本次 clock edge 保存的資料或提交的效果	關鍵 write enable／選擇
2 MemAddr	ALUOut←A+signext(imm)	ALU 選 base 與 immediate
3 MemRead	MDR←Memory[ALUOut]	IorD=1；MemRead=1
4 MemWB	Reg[rt]←MDR	RegDst=0；MemtoReg=1；RegWrite=1
5 MemWrite	Memory[ALUOut]←B	IorD=1；MemWrite=1
6 Execute	ALUOut←A op B，op 由 funct 決定	ALUOp 選 R-type 解碼
7 ALUWB	Reg[rd]←ALUOut	RegDst=1；MemtoReg=0；RegWrite=1
8 Branch	A=B 時 PC←先前保存的 branch target	PCWriteCond=1；PCSrc=branch；ALU 比較 A−B
9 Jump	PC←{當前PC[31:28],IR[25:0],00}	PCWrite=1；PCSrc=jump

多週期資料路徑與控制 （續）

未列出的 architectural write enable 均為0；內部暫存器的寫入依本表模型。Decode 的 opcode 分流為 lw/sw→2、R-type→6、beq→8、j→9，其他 opcode 進入明定的 error transition。狀態3只通往4，不通往5；兩條 memory 路徑分別是讀取後寫回暫存器，或直接寫入記憶體。

關鍵 enable 不是裝飾文字。Fetch state 可 asserted `IRWrite`、`PCWrite`、`IorD=0`；Memory state asserted `IorD=1` 與 `MemRead` 或 `MemWrite`；Write-back state 才 asserted `RegWrite`。Fetch 開始令 oldPC=PC；Fetch 的同一時脈邊緣寫入 IR=Memory[oldPC] 與 PC=oldPC+4。Decode 讀 rs、rt 到 A/B，以已增加的當前 PC 計算 ALUOut=PC+(signext(imm)<<2)，不再把 PC 加4。Branch 相等時才令 PC=ALUOut，不相等保留循序 PC。例 oldPC=0x00400008、imm=-2，Fetch 後 PC=0x0040000c，target=0x00400004；重複加4會錯成 0x00400008。Branch/Jump state 選 next-PC source，不能和一般 `PC+4` update 同時無條件 asserted。

逐週期 CPI 必須由 instruction mix 計算。以 45% R-type、20% branch、20% load、15% store 為例：

$$\mathrm{CPI} = 0.45(4) + 0.20(3) + 0.20(5) + 0.15(4) = 4.0.$$

若每 cycle 300 ps，執行 10^9 instructions 的時間是

$$T = 10^9 \times 4.0 \times 300\,\mathrm{ps} = 1.2\ \mathrm{s}.$$

多週期資料路徑與控制 （續）

同一個假設下，single-cycle 用 800 ps 且 CPI=1，時間是 0.8 s。多週期雖可使用較短 clock，但本題的額外 cycles 超過 clock 縮短比例，因此不一定較快。這個結論只在題目給的元件延遲與 instruction mix 成立；不能把多週期或 pipeline 當成無條件加速器。

FSM 正確性的兩個不變量是：一個 instruction 在同一個 state 不會重複提交 `RegWrite` 或 `MemWrite`；從任何 completion state 回到 Fetch 前，該 instruction 所需的 operand/result state 都已保存。未定義 opcode 要有明確 error transition，不能默默走 default path 並執行任意 memory write。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.5 五階管線模型

五階 pipeline 為：

1. IF：以 `PC` 取 instruction，並計算 `PC+4`。
2. ID：decode、讀 register file，產生 immediate 與 hazard/control 所需欄位。
3. EX：ALU 計算、load/store effective address 或 branch comparison。
4. MEM：執行 data-memory read/write。
5. WB：依 `RegWrite` 寫回 register。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.5 五階管線模型 （續）

每個 pipeline register 是 clock edge 更新的 state snapshot，不是單純導線：

Register	必要 state
`IF/ID`	`valid`、該 instruction 的 `PC`、fetched instruction、`PC4=PC+4`
`ID/EX`	`valid`、該 instruction 的 `PC/PC4`、`rs/rt/rd`、A/B values、sign-extended immediate、control bits
`EX/MEM`	`valid`、該 instruction 的 `PC`、ALU result、store data、destination、`MemRead/MemWrite/RegWrite/MemtoReg`
`MEM/WB`	`valid`、該 instruction 的 `PC`、memory read data 或 pass-through ALU result、destination、`RegWrite/MemtoReg`

`EX/MEM` 若漏帶 `RegWrite` 或 destination register，後面的 WB 就無法知道誰要寫哪個 register。`MemRead` 也不是立即完成：load 在 EX 只產生 address，value 要等 MEM 尾端。這個資料相依是 load-use hazard 的來源。

4.5 五階管線模型 （續）

`PC/valid` 與 operand/control 同屬一條 instruction 的 state，必須一起保持或前進；bubble/flush 令被清除項的 `valid=0`，所有 register/memory writes 都須受 `valid` 門控。Stall 保持 PC 與 IF/ID，在 ID/EX 插入無效項，較舊的有效項仍前進。每階保留自己的 instruction PC，才可在 EX arithmetic fault 或 MEM access fault 記錄正確 EPC；此處顯式存 PC 與 PC4 是教學選擇，也可保存 PC4 並以減4還原 PC。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

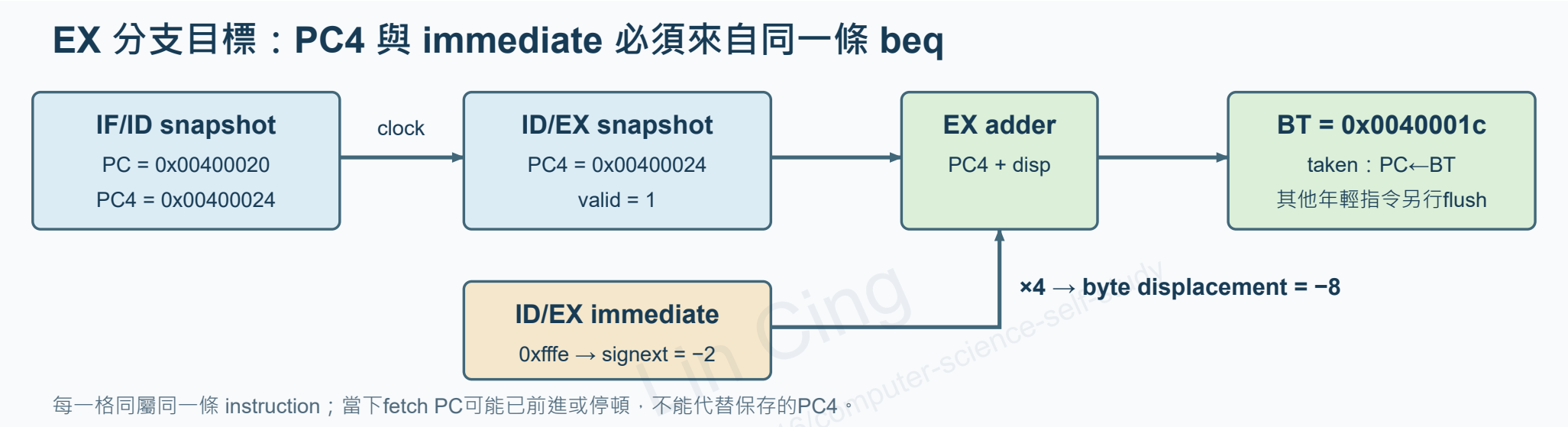
EX 的 branch target 從哪裡來

本模型在 EX 判斷 beq，所以 IF 算出的 PC4 必須先經 IF/ID、再經 ID/EX，不能用當下已在 fetch 年輕指令的全域 PC。32-bit 位址運算為

$$BT = \left(\mathrm{ID/EX.PC4} + 4\,\mathrm{signext}_{16\to 32}(imm_{16})\right) \bmod 2^{32}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EX 的 branch target 從哪裡來 （續）



例 beq 位於 0x00400020、immediate bits=0xfffe。該指令的 PC4=0x00400024，signed immediate=−2、byte displacement=−8，故 BT=0x0040001c。Taken 時下一個 PC 選 BT；not-taken 繼續既定順序路徑。保存的 PC4 不因較年輕指令的 stall/flush 改變，不能以「目前 fetch PC 減固定常數」替代。

無 hazard 時的理想時間

若 stage 完全平衡且 latency 為 t，沒有 hazard 的 n 條 instructions 完成時間為

$$C_{\text{ideal}} = k + (n - 1), \qquad k = 5.$$

Steady state 每個 cycle 完成一條 instruction，但單筆 latency 仍是 $5t$。例如 $n = 100$ 時是 104 cycles，理想 IPC 接近 1。以下長程穩態模型以退休指令數為分母，且各 penalty 是不重疊、不重複計數的實際額外週期：

$$\text{CPI}_{\text{pipe}} = 1 + \text{stall}_{\text{loaduse}} + \text{mispredict_cycles} + \text{structural_bubbles}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pipeline 的理想時間

若 stages 完全平衡，stage latency 是 t，執行 n 條 instructions 的理想 cycles 為

$$k + (n - 1),$$

其中 $k = 5$。Steady-state throughput 是每個 cycle 完成一條 instruction，但單筆 latency 仍是五個 stages。

以 $n = 100$：理想 cycles 為 $5 + 99 = 104$。加速比相對於 ideal single-cycle 為

$$\frac{500}{104} \approx 4.81.$$

實際 single-cycle 的 stage delay 不一定可比，因此這只是相同 stage 假設下的比較。

在本例五級等延遲且忽略 pipeline-register overhead 的模型中，指令數趨於無窮時 speedup 接近 5；單一指令仍依序通過五級，其 latency 並未縮短。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Structural hazard

Structural hazard 發生於兩個同時活躍的 instructions 競爭同一個硬體資源。

五階設計通常把 instruction memory 與 data memory 分開，避免 IF 與 MEM 共用單一 memory port。Register file 的 write 在 cycle 前半或後半、read 在 cycle 前半或後半，可讓同一 cycle 的不同 instructions 合法共享；這需要明確 timing convention。

Register file 通常支援兩個 reads 與一個 write；需要第四個 operand 的 instruction 或更多 concurrent accesses 會增加 ports 與面積。

改 higher associativity、更多 ALUs 或更多 register ports 不是免費；先找出 resource conflict 的頻率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Data dependence

Data dependence 是程式資料流，例如

```
add $8, $9, $10
sub $11, $8, $12
```

`sub` 需要 `add` 產生的 `$8`。True dependence（真相依）不可消除；可透過 forwarding、scheduling 或更多 state 隱藏。

Anti-dependence 與 output dependence 是名稱重用造成的 ordering constraint，例如

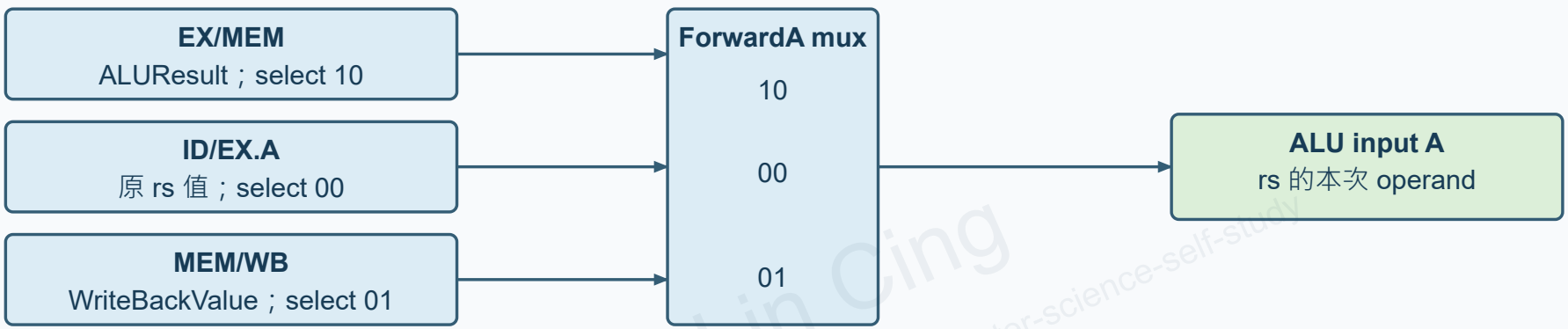
```
add $8, $9, $10
sub $8, $11, $12
```

Register renaming 可把兩個 `$8` 映射到不同 physical registers，消除名稱衝突。

ISA-visible register 名稱相同不等於真資料依賴的 value 是由哪個 instruction 產生。

Forwarding 的 correct path

ForwardA：EX 的 rs 由三個來源選出最新值



EX/MEM match 優先；若最新 producer 是 load，資料尚未就緒，必須先 stall，不能用舊 WB 值。

Forwarding（前送）把 EX/MEM 或 MEM/WB 的 produced value 繞回 ALU input，避免等 WB。

對 `add $8, $9, $10` 後接 `and $4, $8, $5`，EX/MEM 在下一 cycle 已持有 `$8` result，可用 `ForwardA=EX/MEM`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Forwarding 的 correct path （續）

判定必須檢查：

- producer 的 `RegWrite` 已 asserted。
- producer destination 非 `$zero`。
- consumer 的 `rs` 或 `rt` match 該 destination。
- 對 load，EX stage 只有 address，沒有 data；不能把 address 當 loaded value 前送。

對每個在EX使用的source register s，令 E 表示 `EX/MEM.RegWrite && EX/MEM.rd!=0 && EX/MEM.rd==s`，W 表示 `MEM/WB.RegWrite && MEM/WB.rd!=0 && MEM/WB.rd==s`。Mux有三個實際輸入：ID/EX保存的register值、EX/MEM的ALU結果、MEM/WB最終write-back值。選取規則如下。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

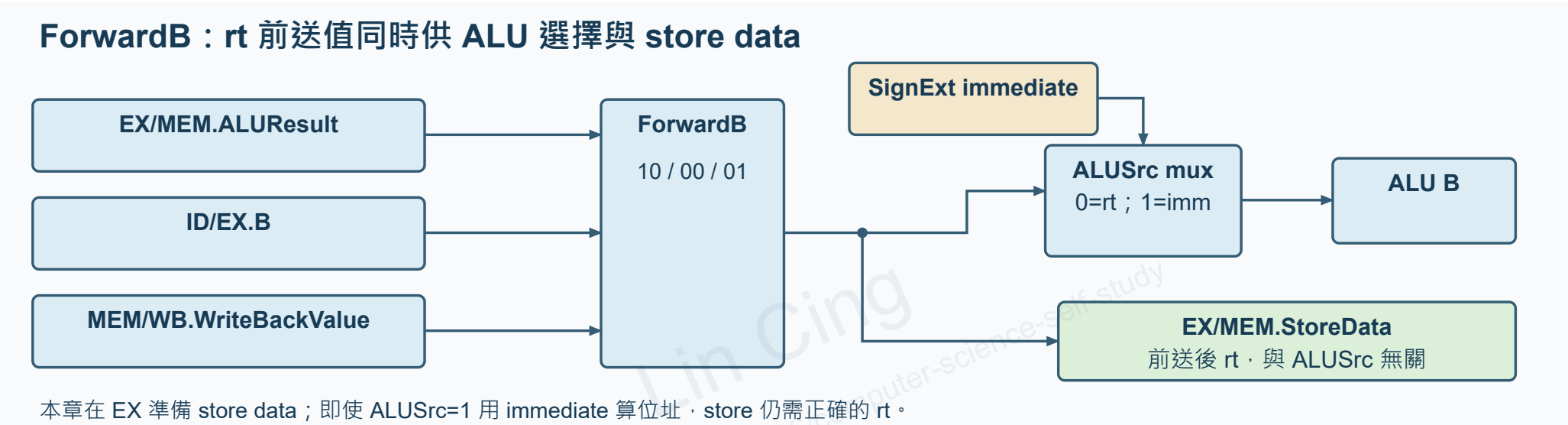
Forwarding 的 correct path （續）

條件	選入ALU的值
E且producer不是load	EX/MEM.ALUResult，編碼10
非E且W	MEM/WB.WriteBackValue，編碼01
非E且非W	ID/EX原值，編碼00
E且producer是load	資料尚未產生，consumer必須先stall，不能前送address或較舊WB值

ForwardA對rs套此規則。ForwardB對作為register operand的rt套此規則，再由ALUSrc mux決定ALU的B輸入是此前送值或sign-extended immediate。Store data也要保存/前送正確rt值。本模型在EX準備store data，所以把緊接load的store-data consumer也保守stall一cycle；另設MEM-stage store forwarding可省此stall，但須另外給路徑與條件。

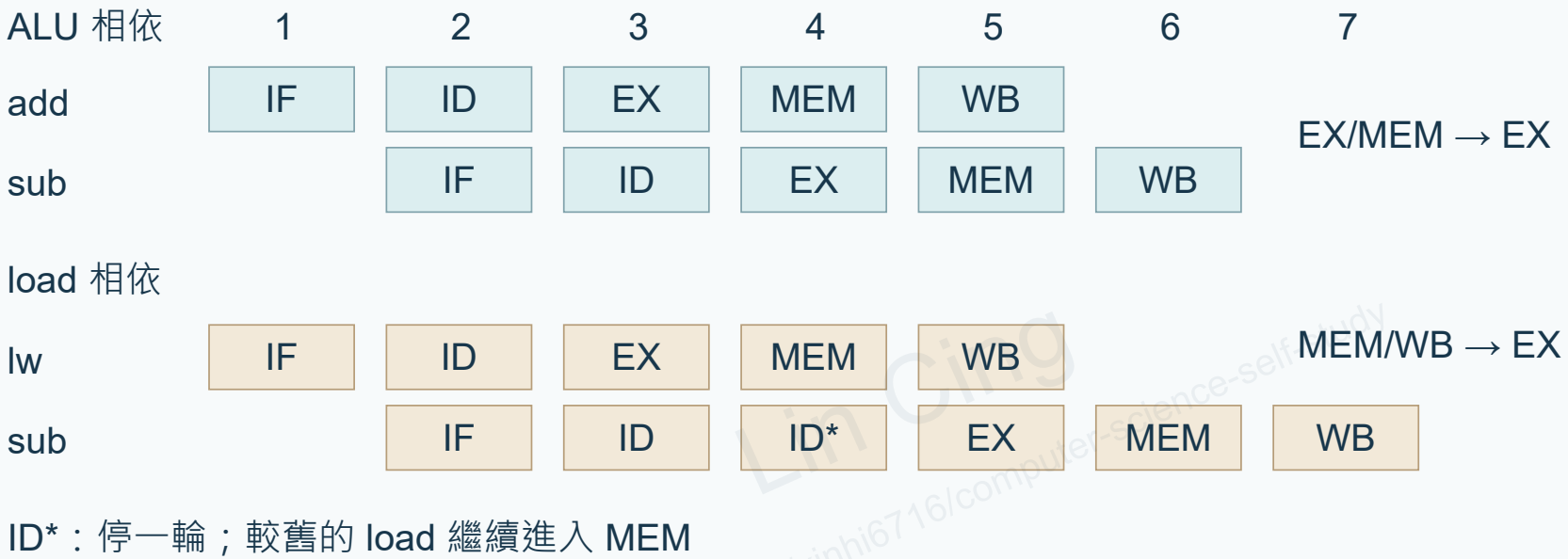
當連續兩個producer都寫s，EX/MEM是較新的定義，必須優先於MEM/WB。例如 `add $1,...; sub $1,...; and $2,$1,$3`，and需要sub結果，不能使用add的舊結果。

Store data 的前送與 ALUSrc



ALUSrc只選擇ALU第二輸入，不決定store寫入memory的資料。Store的address採base加immediate，仍需以最新rt值作data。圖中的分叉點同時送往ALUSrc的register輸入與EX/MEM.StoreData；兩個用途不能混成同一個mux選擇。

兩種相依：為何一個要停、一個不用？



假設五階單發射、單週期記憶體、EX/MEM 與 MEM/WB 可前送到 EX，沒有其他 hazard。上例 `add $s0,$t0,$t1; sub $t2,$s0,$t3`：加法在 C3 EX 結束產生結果，C4 從 EX/MEM 前送至 sub 的 EX，無須 stall。

兩種相依：為何一個要停、一個不用？ （續）

下例 `lw $t0,4($s0); sub $t2,$t0,$s1`：資料在 C4 MEM 結束才就緒，不能在同一 cycle 一開始供 EX。凍結 PC 與 IF/ID 一輪並插入 bubble，sub 在 C5 EX 從 MEM/WB 取得資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Load-use stall

```
lw  $2, 20($1)
and $4, $2, $5
```

`lw` 在 MEM 才取得 data，`and` 下一 cycle 就要在 EX 使用 `$2`。僅有 EX/MEM forwarding 仍沒有 loaded data，因此必須 stall 一 cycle。

Stall 時應：

1. PC 保持原值。
2. IF/ID 保持原值。
3. ID/EX control 清成 no-op，插入 bubble。
4. 讓 `lw` 繼續進 MEM/WB，下一 cycle 前送 loaded value。

若下一條 instruction 不讀 `$2`，不必 stall。比對 `rt` 只適用於會讀 `rt` 的 instruction；`lw` 的 `rt` 是 destination，不能一律當 consumer。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Load-use stall （續）

令 `use_rs/use_rt` 表示IF/ID指令是否在EX需要該source，包括本模型在EX準備的store data。R-type與beq兩者皆1；lw只有use_rs=1；sw兩者皆1；j兩者皆0。對這個subset，load目的暫存器是ID/EX.rt。完整停頓條件為：

```
stall = ID/EX.MemRead && ID/EX.rt != 0 &&
        ((use_rs && ID/EX.rt == IF/ID.rs) ||
         (use_rt && ID/EX.rt == IF/ID.rt))
PCWrite = !stall
IFIDWrite = !stall
IDEXControls = stall ? all_zero : decoded_controls
```

`lw $2,0($1); lw $2,4($3)`不需要stall，因為第二個lw不讀rt。`lw $0,0($1)`不產生可見register結果，所以不因目的為0形成data hazard；memory fault仍須依例外規則處理。一般ISA還需對每個opcode定義source用途與使用stage，不能直接把全部instruction fields都當sources。

逐 cycle 數例：舊值不能代替尚未就緒的新值

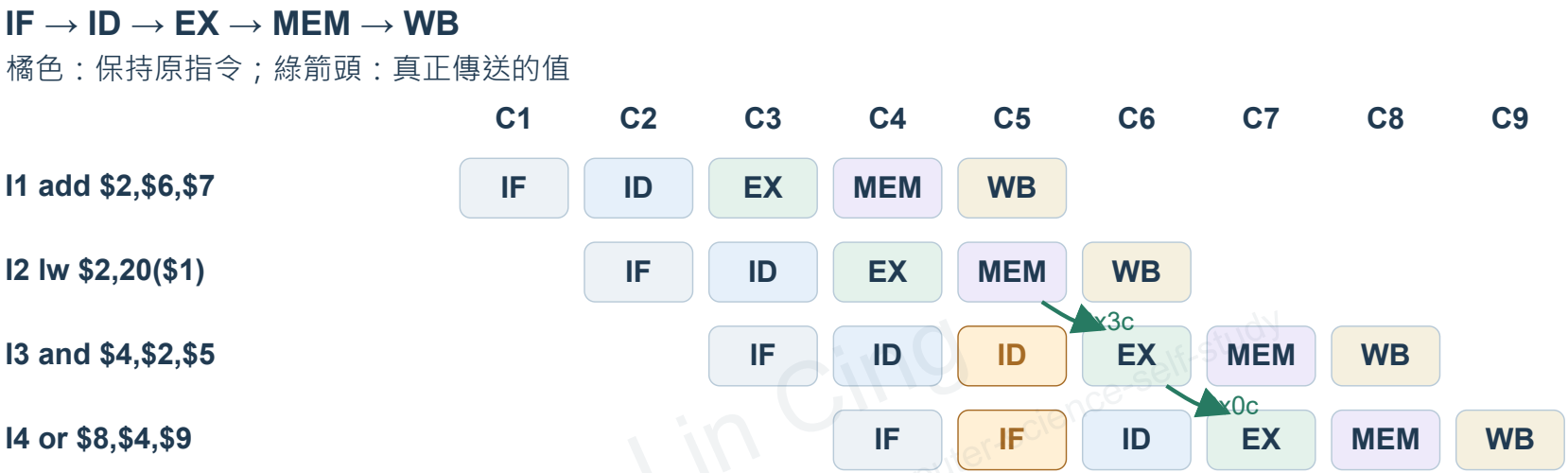
沿用五階、單發射、順序執行的模型；無 cache miss、分支或結構衝突。ALU 結果可由 EX/MEM 或 MEM/WB 前送至 EX；load 資料在 MEM 結束才可供下一 cycle 使用。Register file 在 WB 的前半個 cycle 寫入，ID 後半個 cycle 讀取。

初值：`$1=0x1000`、`$6=0x30`、`$7=0x03`、`$5=0x0f`、`$9=0x80`，且 `Mem[0x1014]=0x3c`；其餘相關暫存器為 0。所有存取對齊且無例外。

```
I1  add $2,$6,$7     # $2 ← 0x33
I2  lw  $2,20($1)    # $2 ← 0x3c，覆寫 I1 的值
I3  and $4,$2,$5     # $4 ← 0x0c
I4  or  $8,$4,$9     # $8 ← 0x8c
```

`I3` 必須讀到最近一次寫入 `$2` 的 `I2`，不是更早的 `I1`。相依關係是 `I2 → I3 → I4`；`I1` 與 `I2` 的目的暫存器相同，但 `I2` 不讀 `$2`，不構成兩者之間的 RAW 相依。

九個 cycles：停住前端，較舊指令繼續前進



C4：偵測相依 → C5：EX bubble（valid=0），I3 留在 ID，I4 留在 IF。

C6：MEM/WB → I3.EX；C7：EX/MEM → I4.EX。I4 在 C9 完成。

每格表示該 cycle 內所處的 stage，cycle 結束才跨越 pipeline register。C4 偵測到 `I2.EX` 的 load 與 `I3.ID` 相依；在 C4 結束的 edge 保持 PC、IF/ID，並向 ID/EX 寫入無效項。因此 C5 沒有有效指令在 EX，並非把 load 停在原處。

從時序圖核對控制訊號與數值

Cycle	指令位置與可用值	控制或運算結果
C4	I2.EX 算出位址 `0x1014`；I3.ID 需要 `$2`	`PCWrite=0`、`IFIDWrite=0`；下一個 ID/EX 為無效項
C5	I1.WB 寫 `$2=0x33`；I2.MEM 在末端取得 `0x3c`；I3 仍在 ID	I3 讀到並暫存舊值 `0x33`；此時 hazard 已解除，前端可在 edge 前進
C6	I2.WB 的值為 `0x3c`；I3.EX	`ForwardA=01` 選 MEM/WB，覆蓋暫存舊值；`0x3c AND 0x0f=0x0c`
C7	I3.MEM 帶著 ALU 結果 `0x0c`；I4.EX	`ForwardA=10` 選 EX/MEM；`0x0c OR 0x80=0x8c`

若略過 C5 的 bubble，I3 會在 C5 的 EX 過早取值：最新 producer 的 EX/MEM 只有 load **位址** `0x1014`，MEM/WB 的 `0x33` 則是舊版本。兩者都不能代替尚未產生的 load 資料；錯用舊值會得到 `$4=0x03`。

有、無 forwarding：比較的是同一段程式

模型	I3 的 EX	I4 的 EX	最後 WB	額外停頓
有 EX/MEM 與 MEM/WB 前送	C6	C7	C9	1 cycle
完全無前送，只能從 register file 讀	C7	C10	C12	4 cycles

無前送時，I3 在 C4、C5 的 ID 被擋住，直到 C6 的 WB 寫入新 $2 後才能讀取；I4 則在 C7、C8 的 ID 等待，直到 C9 的 WB 寫入 $4。由於允許同 cycle 先寫後讀，不再額外等待一個 cycle。

四條有效指令的理想完成時間為 4+4=8 cycles；上述兩模型分別為 8+1=9 與 8+4=12。有限序列的 CPI 為 9/4 與 12/4，包含填滿與排空成本，不能把它直接當成長程式穩態 CPI。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Control hazard、branch delay 與 flush

Branch 改變 fetch path。五階 pipeline 在 branch 尚未到 EX 前，IF/ID 已抓入較年輕 instructions；若 taken 且 prediction 是 not taken，就必須丟棄 wrong-path instructions。

branch penalty 由 resolve stage 與 predictor 決定，不等於 architectural delay slot。若要估計 per-instruction cycle cost，可寫成

$$\text{mispredict_cycles} = f_{\text{branch}} \sum_{r} P(\text{wrong}_r \mid r) C_r.$$

例如 20% instructions 是 branches，其中 60% taken，predictor 一律 not taken，且 taken 在 EX 解出後 flush 兩個 younger instructions：

$$0.20 \times 0.60 \times 2 = 0.24 \text{ cycles/instruction}.$$

Classic MIPS 的架構另有一個 branch delay slot。`beq` 後緊接的 instruction 無論 taken 與否都屬於 architectural execution；taken 時只 flush delay slot *之後* 的 wrong path。若在 EX 解出時，delay slot 在 ID，下一條在 IF：

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Control hazard、branch delay 與 flush （續）

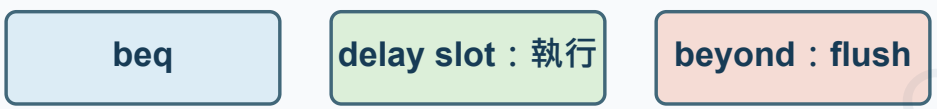
分支在 EX 解出時的 flush 與 architectural delay slot

無 architectural delay slot



taken：IF.Flush=1、ID.Flush=1、PC←target

classic MIPS 的一個 delay slot



taken：delay slot 必須執行；只 flush 再下一條 wrong-path

Architectural delay slot 是 ISA 語意；branch resolution penalty 是 microarchitecture 成本，兩者不可混為一談。

No-delay-slot model 需 flush ID 與 IF 的兩個 younger instructions；one-delay-slot model 只 flush delay slot 之後的 IF instruction，slot 必須照 ISA 語意執行。不能把「flush 幾個 stages」和「delay slot 是否執行」混成同一條規則。

預測器可以早於 EX 做 direction guess，但事件順序仍是 prediction、fetch、resolve、misprediction recovery。BTB hit 只表示有 target 可用，不表示 direction 一定正確。

Pipelined CPI 的完整邊界

以同樣 20% branch、60% taken、one-cycle load-use bubble 出現於 10% instructions 的模型：

$$\mathrm{CPI} = 1 + 0.20(0.60)(2) + 0.10 = 1.34.$$

這個 1.34 只描述題目明確列出的 penalties。Structural hazard、cache miss、branch target miss、long-latency functional unit 與 front-end limit 都必須各自加入，不能以「五階所以 CPI=1」抹去。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic branch prediction

2-bit saturating counter 可抵抗單次雜訊。狀態例如 strongly not taken、weakly not taken、weakly taken、strongly taken；實際 outcome 為 taken 時向 strongly taken 更新，not taken 時向 strongly not taken 更新；更新方向不由本次預測正確或錯誤單獨決定。

Correlating predictor 以 global history 預測具相關性的 branches。Tournament predictor 依不同 predictor 的歷史表現選擇。

Branch target buffer（BTB，分支目標緩衝）快取 target；prediction 仍要同時判斷是不是 branch 以及 taken direction。BTB miss 不等於 direction prediction 錯。

測試需覆蓋 cold start、alternating pattern、correlated branches 與 alias。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.9 Exception 與 interrupt

Exception 是由目前 instruction 引起的同步事件，例如 overflow 或 undefined instruction。Interrupt 常指外部非同步事件，例如 I/O request。不同 ISA 的命名不盡相同。

Pipeline exception handling 要保存 faulting PC、cause 與必要 address，跳至 handler。Handler 修復或終止 process 後，若可恢復則由 exception return instruction 回到 architecture-defined PC。

Handler 執行前的 architectural state 必須符合 ISA 規定。若較早 instruction 已 commit、較晚 instruction 被 flush，程式仍可建立單一一致的 interrupted state。

先分清 precise exception 與只保證 imprecise machine check 的差異。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Precise exception 的週期與狀態

在單發射五階 pipeline，令 faulting I4 在 EX 發現 arithmetic overflow。此 cycle 的合法 stage snapshot 是 I2 在 WB、I3 在 MEM、I4 在 EX、I5 在 ID、I6 在 IF；I1 已在更早 cycle commit。I2、I3 沒有 exception；本例的 I3 是在 WB 提交暫存器結果的非 store 指令；I4 與較年輕的 I5、I6 必須阻止 architectural writes。

精確例外：exception buffer、older-instruction drain 與 redirect

cycle	WB	MEM	EX / control	ID	IF
cycle T	I2 commit	I3	**I4 fault**	I5	I6
cycle T+1	I3 commit	—	pending	—	PC held
cycle T+2	—	—	redirect PC	—	handler fetch

ExceptionPC / Cause / BadVAddr latch；停止 younger writes

Older writes drain 後才 redirect；I4/I5 不得留下 effect

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Precise exception 的週期與狀態 （續）

Cycle	WB	MEM	EX / control	ID	IF
T	I2 commit	I3	I4 arithmetic fault	I5	I6
T+1	I3 commit	—	exception pending	—	PC held
T+2	—	—	PC←handler，送出保存的 cause	—	handler fetch

本表以每 cycle 開始後的 stage 活動為準，commit 在該 cycle 尾完成：I3 在 T+1 尾提交後，controller 在 T+2 開始選 handler PC，首條 handler 指令於 T+2 的 IF 取得，尚未進入 ID。

在 T 的同 cycle control 中，硬體記下 ExceptionPC=I4.PC、cause 與依例外類型必要的位址（算術 overflow 不一定有 bad address），deassert I4 與 younger writes，並 flush I5、I6。Precise exception 的契約是 handler 看到的所有 architectural effects 等於依序執行完 I1、I2、I3，且 I4 與更晚的 register/memory writes 完全不出現。為了讓較舊的 I2、I3 先完成，controller 可先把 exception 放進 exception buffer，停止 fetch 並 drain older stages；I3 在 T+1 commit 前，handler 不能先看到它的 effects。

Precise exception 的週期與狀態 (續)

若有多個 exceptions 同 cycle，必須由 spec 指定 priority。通常採用最舊 instruction 的 synchronous exception；但 asynchronous interrupt、machine check 與 precise/imprecise policy 可能不同。Handler 入口前也要保存/切換 privilege、status、EPC、cause；僅在例外類型需要時保存 bad-address registers。

Out-of-order execution 可用 ROB 完成相同效果：年輕 instruction 可先執行，但 commit 必須依 program order。`I4` 成為 ROB head 且之前 `I1` 至 `I3` 都 commit 後，才丟棄 `I4` 與所有較年輕 entries，再清空 speculative rename mapping 並 redirect。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.10 ILP 與 multiple issue

Instruction-level parallelism（ILP，指令層平行）是同一執行流中可重疊的 independent instructions。Pipeline 在 stages 間重疊 instruction；multiple issue 在同一 cycle 啟動多條。

Static multiple issue 由 compiler 把 instructions 排成 issue packets。VLIW（very long instruction word，超長指令字）把多個 operations 編碼在 fixed packet，硬體控制較簡單，binary portability 取決於 ISA/model。

Dynamic multiple-issue 由硬體在 runtime 從 instruction window 選 ready instructions。它可跨越 compiler 未預期的 latency，但需要 scoreboard、wakeup/select、ROB 等大量狀態。

峰值 IPC 4 表示理想寬度，不等於所有程式都能維持 4。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Superscalar 與 dynamic scheduling

Superscalar（超純量）在一個 cycle 嘗試 fetch/decode/issue 多條 instructions。Issue logic 檢查 operand readiness 與 functional-unit availability；out-of-order execution 可讓後續 ready instruction 先執行。

Speculation 在 branch outcome 未知時沿預測路徑執行。錯誤預測時，ROB 或 checkpoint 丟棄 younger results，register mapping 回到正確狀態。

Register renaming 把 ISA registers 映射到更多 physical registers，消除 false dependencies。它不改變 architectural register 的可見數量。

OoO execution 不等於 OoO commit。很多處理器以 in-order commit 保存 precise exception。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

亂序執行、rename 與 ROB 的逐 cycle 追蹤

固定單一 dispatch/cycle、單一 commit/cycle，依 program order 配置 ROB 與新 physical destination；每條指令最早在 dispatch 的下一 cycle 開始執行。獨立 load unit latency=4 cycles，pipelined integer ALU latency=1 cycle。結果在 cycle 結束可用，只能從下一 cycle 供 consumer 或 commit。無 cache miss、branch、fault 與其他 structural hazard；ALU 不溢位。

```
I1: lw  r1,0(r2)
I2: add r3,r1,r4
I3: add r5,r6,r7
I4: add r1,r8,r9
```

初始 r1/r2/... 對應 p1/p2/...。Rename 逐條配置 I1→p11、I2→p12、I3→p13、I4→p14。I2 在 dispatch 時已把 r1 source 綁為 p11；I4 雖把 speculative r1 mapping 改成 p14，也不能改寫 I2 保存的 source tag。RAW I1→I2 仍存在；I1/I4 的同名 destination 與 I2/I4 的名稱衝突由不同 physical registers 分開。

亂序執行、rename 與 ROB 的逐 cycle 追蹤 （續）

執行結果可亂序，commit 仍依 I1 → I2 → I3 → I4

	1	2	3	4	5	6	7	8	9
I1: lw → p11	D	LD1	LD2	LD3	LD4	C			
I2: add → p12		D	wait	wait	wait	EX	C		
I3: add → p13			D	EX	ready	ready	ready	C	
I4: add → p14				D	EX	ready	ready	ready	C

D：dispatch；EX/LD：執行；wait：等operand；ready：等ROB head；C：commit

I2 source = p11 (I1); I4 destination = p14. Result end of cycle → use next cycle.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

亂序執行、rename 與 ROB 的逐 cycle 追蹤 （續）

指令	Dispatch	執行	結果 ready（cycle 尾）	Commit	原因
I1	1	load cycles2–5	5	6	ROB head ready後才能提交
I2	2	EX cycle6	6	7	讀 p11，必須等 I1 結果
I3	3	EX cycle4	4	8	ready仍等較舊I1/I2提交
I4	4	EX cycle5	5	9	新p14不覆寫I1的p11

例如取 memory[0(r2)]=10、r4=1、r6=2、r7=3、r8=4、r9=5，I3先算得p13=5，I4算p14=9，I1算p11=10，I2最後讀p11得p12=11。最終 architectural r1=9、r3=11、r5=5，等同程式順序；若I2錯讀p14，r3會錯成10。Ready、executed與committed是三種不同狀態。

另取 r4=2147483647、保持 I1 的 load 值10，I2 的 signed add 在 EX cycle6 發現 overflow；cycle6仍可先commit較舊I1，I2記下fault但不commit。輪到I2為ROB head時，捨棄I2/I3/I4，恢復到committed rename map，r1對應p11而非p14；handler看到r1=10，r3/r5仍是執行本段前的architectural值。已算出的p13/p14沒有對程式提交，不能以「先執行完」為由保留。

亂序執行、rename 與 ROB 的逐 cycle 追蹤 (續)

這個模型未處理 speculative stores、memory disambiguation或分支checkpoint；加入它們時還需 store queue、例外與恢復規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多發射資源邊界

設理想每 cycle 可 issue 4 instructions，但程式中只有 20% 的相鄰指令 independent，其他時刻只能 issue 1。若 workload 的 instruction energy 與 fetch bandwidth 不變，平均 IPC 應以動態 issue 分布計算，不能把峰值直接乘上 4。

限制可能來自：

- data/branch dependencies。
- instruction and data cache bandwidth。
- branch predictor accuracy。
- functional-unit mix 與 register ports。
- reorder buffer size。
- power density 與 thermal constraint。

分析 workload 的 IPC histogram，比只報 peak IPC 更能解釋真實效能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.11 Opteron X4 案例

Opteron X4（Barcelona）fetch x86 instructions，decode/translate 成 AMD 所稱的 RISC operations（Rops），再由 dynamic scheduled speculative pipeline 執行。本章案例模型描述最多每 cycle issue 3 Rops，整數 pipeline 約 12 stages，浮點 execution queue 更長。

可能瓶頸包括：

- complex x86 instruction 拆成多個 Rops。
- hard-to-predict branches 造成 restart。
- 長依存或 cache misses。
- translation/rename/ROB 等 front-end 資源。

這些歷史數字用於理解 microarchitecture，不能當成 2026 年任何現行 CPU 的規格。

此案例的 issue 寬度及 pipeline 深度只適用於上述 Barcelona 模型；換架構時需重新取得前端、執行端與記憶體限制，不能由歷史峰值推算現代程式時間。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

4.12 HDL 與模擬

Hardware description language（HDL，硬體描述語言）描述 module 的 ports、state、combinational equations 與 clocked behavior。Verilog 的 `always @(posedge clk)` block 表示 clocked process；blocking 與 nonblocking assignment 的語意不能混用而不檢查。

HDL 模擬只能證明 testbench 覆蓋範圍內的行為。Synthesis、timing analysis、formal assertion 與實機測試回答不同問題。

以時序 assertion 可以檢查 hazard、flush 與 commit order。Simulation 覆蓋到的 testbench 案例、synthesis 結構、static timing 與實機量測各自回答不同問題，不能互相取代。

可檢查的時序條件包括：stall 且沒有 redirect 時，下一個 edge 的 PC 與 IF/ID 必須保持；valid=0 的指令不得寫 architectural state；faulting instruction 與其後的指令不得 commit；ROB commit 的指令序號必須依 program order 遞增。這些條件分別對應停頓、bubble 與 precise exception 的正確性。

4.13 設計誤解

「Pipeline 能降低每條 instruction latency」不成立。Balance 的五階 pipeline 讓每條 instruction 經五個 stages；收益來自許多 instructions 重疊。

「Forwarding 可解決所有 data hazards」忽略 load-use、long-latency units 與真資料依賴。

「Pipeline 設計與 technology 無關」忽略邏輯深度、clock distribution、power 與 memory latency 的變化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.14 章末模型收束

設計一個 processor 的核心檢查是：

1. 逐段驗證每條 instruction 完成後的 architectural state。
2. 列出每個 stage 的資源與結構衝突。
3. 證明 branch 與 exception 能恢復正確狀態。
4. 在相同 workload 下報告 latency、throughput、IPC、frequency 與 energy。
5. 測試資料依賴、memory access、wrong-path behavior 與邊界案例。

Single-cycle 易理解但 clock 被最長 path 支配；五階 pipeline 改善 throughput；forwarding、stall、prediction、precise state 與 dynamic scheduling 處理重疊帶來的代價。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.15 處理器技術演進

從單週期到多週期與管線，主要改變是把長組合路徑分配給不同時段或不同指令，並用暫存狀態追蹤尚未完成的工作。管線重疊引入資料與控制危害；前送和停頓維持值的正確性，分支預測則用恢復成本換取前端不中斷。

動態排程及暫存器重新命名讓後續就緒指令繞過尚未完成的運算，但真正資料相依仍須等待。亂序執行配合依序提交，把執行效率與精確例外的程式順序契約分離。更寬發射與更大視窗也增加選擇邏輯、線路、能耗與驗證成本，所以歷史進展不能只用管線階數或峰值IPC排序。

比較每個時代採用技術所解決的限制、增加的狀態，以及出錯時必須恢復的狀態，避免把歷史案例當成現行處理器規格。

參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 4. CSO course slides, processor datapaths and pipelines.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

4.15 處理器技術演進 （續）

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study